行动闭环的两端

AgentCore 让 Agent 敢行动，Agentic工作台让结果看得见

演讲嘉宾：黄达

目录

CONTENT

01 开场：边界在哪里

02 AgentCore：敢行动的生产级底座

03 Agentic工作台：看得见的最后一公里

04 延伸：行为洞察→主动干预

05 场景实践与收尾

01 开场：边界在哪里

从分析闭环，走向行动闭环

关于我

黄达

亚马逊云科技 · 大中华区解决方案架构团队

合作伙伴解决方案架构师（PSA）· 负责与数数科技（ThinkingAI）共建联合方案

关注方向

· 生成式 AI 与 Agent 在企业生产环境的落地
· 与 ISV 合作伙伴共建联合方案、PoC
· AI-DLC 在软件企业中的实践

广度

先后服务于 **40+** 家国内外 ISV 合作伙伴

其中很多来自游戏与数据分析领域

帮助多家合作伙伴与 AWS 联合开发 Agent 产品

积累

15 年	4 年	2 年
软件研发	云原生解决方案	AI 智能体项目

Data Agent 的闭环，卡在哪里？

从分析闭环，走向行动闭环

中间段：大家都在做深

左端：不敢行动
权限失控·调用跑飞
事后无法审计

→ 感知 → 分析 → 决策 → 执行 →

右端：看不见
结果停在技术侧
业务用不上

行业信号：不是不想用，是不敢放手

43% → 27%

对完全自主 Agent 的信任度，一年内的变化（凯捷研究院，1,500 位高管调研）

真实场景

Agent 分析做得很好，但没人敢让它直接改线上配置；或报告做出来了，业务方没打开过

行动闭环真正卡住的不是中间，而是两端——不敢行动，和看不见。

来源：Capgemini Research Institute，2025-07 新闻稿（调研于 2025 年开展，样本 1,500 名高管）

02 AgentCore：敢行动的生产级底座

Amazon Bedrock AgentCore · 闭环的第一端

Amazon Bedrock AgentCore：五项核心能力包住一个 Agent

各组件完全解耦，可单独使用也可任意组合；不绑定特定框架或模型（官方服务清单不止这五项：另有 Policy、Harness、Evaluations、Browser、Code Interpreter、Registry 等）



模块化：每项能力可以单独用，也可以组合用；不绑框架，不绑模型。按需组合示例：只用 Runtime 做托管运行（不接 Gateway / Identity）；或 Gateway + Identity 做工具调用治理（不接 Memory）。

Runtime：每个会话一台独立的 MicroVM

会话隔离示意



硬件级隔离：会话结束，整台销毁、内存清理

单次运行的生命周期（默认值，均可配置）



按需弹性	从零扩缩，不用预置；等模型回复的 I/O 时间不计 CPU 费
状态默认临时	跨会话要留下的上下文，交给 Memory
不绑框架	任意框架、任意模型，支持 MCP / A2A

在生产环境里，隔离不是选项，是默认值。

Identity：谁让它动的、它拿谁的钥匙？



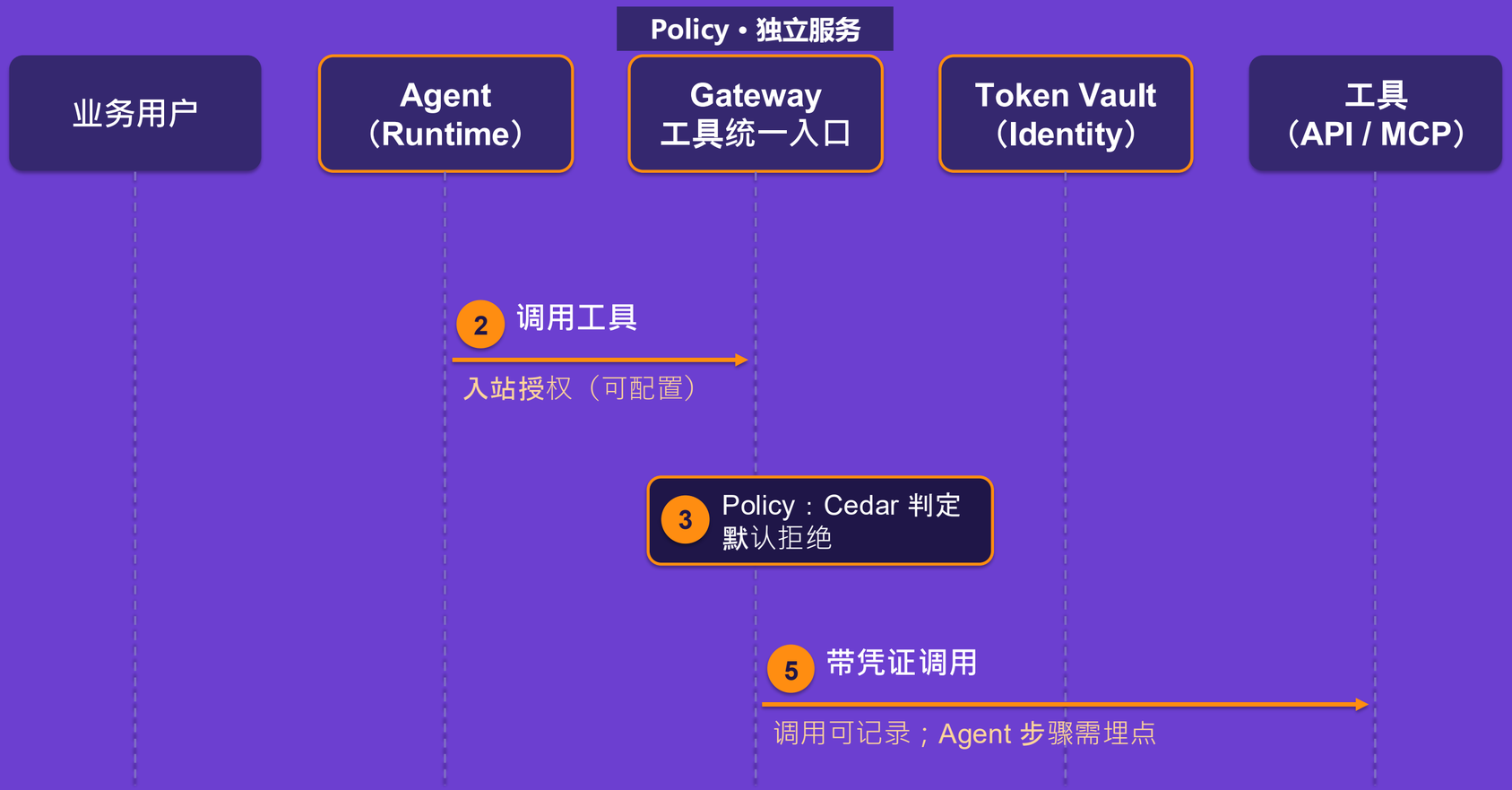
谁让它动的？

入站认证：Runtime、Gateway 各自配置 IAM 或 JWT，对接企业现有身份提供方

它拿谁的钥匙？

Token Vault：KMS 加密；凭证只给最初取得它的“Agent + 用户”组合

Gateway：工具调用入口 | Policy：独立的规则判定服务



这个动作允不允许？

Policy 是与 Gateway、Memory、Runtime 等并列的独立 AgentCore 服务（官方架构同级列出），不是 Gateway 的子功能。

它的拦截点挂在 Gateway 的调用路径上：每次工具调用执行前先判定，规则写在 Agent 代码之外，默认拒绝。

身份跟着用户走，工具统一从网关进，规则写在代码外面。

Memory：记得住，更要记得准

AgentCore Memory 托管的两层记忆

短期记忆

按会话逐轮记录原始事件

事件按设定期限自动过期（3–365 天）

记忆策略：后台异步提炼

长期记忆

提炼偏好 / 事实 / 摘要，跨会话保留，按相关性检索

也支持通过 API 直接写入

可按策略配置最少 31 天的保留期（到期自动清除）；不配置则不过期

数数知识库与 Skills

- **沉淀企业指标口径、业务规则、工作方法和实践经验，并复用**
- **口径示例：付费率怎么算、哪些渠道算买量**
- **规则示例：业务口径与数据治理规则写在知识库里**
- **方法示例：数十个行业 Skills，如 LTV 曲线、付费归因**

两种设计风格互补，AgentCore Memory实现了大部分Agent需要的记忆的底层需求。但不影响企业继续实现自己的独特逻辑。

Memory：怎么接、为什么直接用

接入方式与模型、框架解耦；开发者可以从 API、SDK 或现有运行环境直接使用。

怎么接

01 数据面 API
CreateEvent、ListEvents
RetrieveMemoryRecords、BatchCreateMemoryRecords

02 SDK / CLI / 控制台
Python SDK、Strands Agents、AgentCore CLI、AWS 控制台、Boto3

03 运行位置不限
Agent 不在 Runtime 内，也可以接入 Memory

为什么直接用托管 Memory

提炼策略
用户偏好、语义事实、会话摘要、Episodic，也支持自定义策略

多租户隔离
namespace 按 actorId / sessionId 分层，可前置 Gateway + OAuth 与细粒度访问控制

治理
KMS 客户托管密钥、执行角色、元数据过滤与记忆记录流式投递

开发者原本要自己处理的提炼、隔离、加密和权限控制，被封装成可直接接入的托管能力。

AgentCore harness：配置优先的编排循环

Harness 运行在 AgentCore Runtime 中，提供开箱即用的编排循环；Runtime 则让团队自己编写编排逻辑。

Harness | 配置优先

模型：Bedrock / OpenAI / Gemini / LiteLLM

工具：Gateway、MCP、Browser、Code Interpreter

记忆：短期 + 长期，跨会话保留

身份：Identity

Runtime | 代码优先

每个会话有状态，运行在独立 microVM

自带文件系统和 shell，可写代码、执行代码

不单收 Harness 费用，按底层能力实际用量计费

状态：GA

配置快速起步 → 配置不够用时导出 Strands 代码 → 部署到 Runtime 继续运行

版本发布与 A/B

evaluations + optimization 可按会话测试 Agent 新版本，并输出显著性报告；immutable versions + named endpoints 支持安全发布和快速回滚。这里测试的是 Agent 版本，与后文面向用户运营干预的 A/B 不同。

Observability + Evaluations：看见过程，量化质量

一次请求的执行追踪 会话 › 调用链 › 步骤

示意图



接入 OpenTelemetry 埋点后，每一步的耗时、输入输出、工具调用都能回放

Observability

指标：会话数 · 延迟 · 时长 · Token 用量 · 错误率

默认进 Amazon CloudWatch，兼容 OpenTelemetry，可接现有监控工具；harness 里的每一步动作也自动进入这里的追踪视图，不需要额外埋点

Evaluations

大模型当裁判：提供一组内置评估器（正确性、工具选择准确率、目标达成……），其中含 2 个专门的 skill evaluators（技能选择准确性、技能指令遵循），并支持自定义

三种方式：在线抽样 · 按需 · 批量回归

评的是 Agent 本身的质量

决定 Agent 能不能上生产的，不是模型能力的上限，而是失控风险的下限。

案例 1：LobeHub — 本地 Agent 升级为云端平台

互联网 · 多智能体协作 AI 平台

01 本地 Agent

能力主要运行在客户端

设备性能与环境差异带来稳定性不足、资源占用高和执行效率不可预测

多 Agent 协作、并发、状态管理和统一运维受限

02 AgentCore

Runtime
把 MCP Tool 从客户端迁移到云端托管

Code Interpreter
把编写代码和访问工具封装成内置沙箱能力

03 云端平台

从“应用”升级为“平台”

能力成为可编排的基础设施

构建 Agent harness 更快、更可靠，基础设施维护成本更低

> **提升了构建 Agent harness 的速度和可靠性，也显著降低了基础设施的维护成本。**

来源：https://aws.amazon.com/cn/solutions/case-studies/lobehub/

案例 2：Mattel163 — 玩家洞察平台

游戏出海 · 手游玩家反馈分析

改造前

分析师人工阅读问卷、访谈与多语种反馈

一份玩家洞察报告需要数天

数天 → 分钟级



改造后

AgentCore Runtime + Strands Agents

多智能体平台分钟级生成玩家洞察

底座

会话隔离与弹性伸缩交给 Runtime

数据

多语种、多来源数据先治理再交给 Agent

交付

结果直接给发行与运营团队使用

给游戏团队的启发：从高频、低风险的分析场景开始，再逐级放权。

组件：AgentCore Runtime（框架使用 Strands Agents）

来源：https://aws.amazon.com/cn/blogs/china/mattel163-intelligent-platform-generation/

案例 3：Cox Automotive —— 不到一年，17 个智能体方案投入生产

汽车互联网平台

不到 1 年

从零经验到规模化落地

17 个

智能体方案投入生产

8–48 小时

维修估价缩短到 30 分钟

AgentCore Memory

在整个采购与服务流程中保持上下文连续（公开案例原文：maintains seamless context throughout the buying journey）。

口径边界

公开案例只明确提到 Memory，其余具体组件未进一步细分；成果来自 Agent 方案与 AgentCore 平台便利性的综合作用，不单独归因于某个组件。

来源：https://aws.amazon.com/solutions/case-studies/cox-auto-case-study/

更多真实案例：规模化生产的共同打法

案例	行业 / 组件	核心数字	说明
蓝色光标	广告营销 · 「内容宇宙 Studio」· AgentCore Runtime	**制作效率翻倍；单条视频 40 分钟 → 20 分钟**	端到端视频生成智能体 DDD，单次活动稳定处理超 3,000 条中长视频任务，制作成功率超过 99%。
猎豹移动 × 聚云「CostQ」	软件行业 · Runtime + Gateway	**3 个月 → 5 周；运维工作量减少约 80%**	将云成本 API 封装为标准化 MCP Server，经 Gateway 统一调用，自然语言即可完成成本归因分析。
Druva	数据安全 SaaS · AgentCore	**68%；问题解决提速 58%**	68% 的支持问题无需人工介入；DruAI 多智能体跑在 AgentCore 上。
Ericsson	电信行业 · AgentCore	**实现双位数提升；覆盖数万名员工**	3G/4G/5G/6G 系统横跨数百万行代码、数千个互联子系统；AgentCore 融合数据与信息，交付能力空前的 AI Agent。
BNB Chain	Web3/区块链 · 「BNB Agent Studio」· Runtime	**数周 → <15 分钟；成本管理更高效**	面向开发者的 Agent 托管平台，核心层部署在 Runtime（MicroVM 隔离）。
易点天下 Yeahmobi	出海营销 · 「EC-Agent」· Runtime 等 5 项能力	**开发成本 -80%；创意效率 10 倍以上**	单个优化师可管理的广告投放活动数从 30 个增至 300 个；2 周完成从 EC2 到 AgentCore 的整体迁移。
紫讯「BrowserAct」	浏览器自动化 · Runtime	**0 等待体验；用户基数半年 +300%**	秒级生成运行环境 + 预热机制实现即时调度，按 CPU 实际用量计费，99% 高可用。

共同点：把生成、分析、成本归因、客服、系统知识融合这类生产级的“脏活”交给 AgentCore，团队才能专注核心业务，从数周到分钟级、从人工到自动完成规模化。

来源：蓝色光标、猎豹移动 × 聚云 CostQ、BNB Chain、紫讯 BrowserAct 为 AWS 官方页面；Ericsson 来自 AWS AgentCore GA 官方博客；Druva 为 AWS Case Study；易点天下来自 AWS 客户案例及 CBISMB 媒体报道。

03 Agentic工作台：看得见的最后一公里

数数 Agentic Engine × 智能体工作台：让结果看得见

Agentic工作台整体架构：能力在数数，入口在工作台

数数 Agentic Engine 已把数据的商业价值讲清楚；本章聊它之外的场景：刁钻的分析角度、还没定型的 AI 工具想法，不动研发，先用 Amazon Quick 看到效果。

数数 Agentic Engine

能力层 · 算对、算快

· 数据采集 → 分析引擎 → 运营策略 → 效果验证
· 实时数据 → 实时决策
· 通过全链路 CLI / MCP 开放给任意 Agent
· 数十个行业 Skills

MCP / CLI

智能体工作台

前端 · 交到业务手里

· 如 Amazon Quick（专为工作打造的 AI 伴侣），也可以是 Claude Code、Kiro 等桌面 Agent
· 业务用户用自然语言提问、看图、追问、分发
· 其他能力：研究、自动化、应用
· Quick：定制化需求、早期场景的快速验证

业务人员提出问题 → 工作台经 MCP / CLI 调用数数 Agentic Engine 的数据、口径与 Skills → Amazon Quick 呈现看板与结论 → 业务人员追问、分发、反馈。
能力在数数 Agentic Engine，入口在智能体工作台：Agentic Engine 负责算对，工作台负责交到业务手里。

实时数据 + Skills 进入工作台

数数实时链路

01 多端实时上报 → **02 秒级查询** → **03 信号识别** → **04 动作触发**

“Dashboards tell you what broke yesterday; agents fix it today.”

工作台里看见正在发生的

· 直接问当前数据

· 对话中生成可交互图表与报表

· 定时简报与推送

Skills 进入工作台

Skills 是企业级 Agent 可调用的后台能力，与 MCP server 属于同一类工具。

示例：实时分析、留存与 LTV、用户分层、埋点方案、A/B 实验和用户触达。

说明：实测使用演示项目中的固定区间数据，不是实时流。

调用链路示意

Amazon Quick 经 MCP / CLI 调用数数 Agentic Engine



数据口径

分析是数数 Agentic Engine 算的，口径是数数的；Quick 负责把结果交到业务手里。

MCP 是主通道（标准协议接工作台 / AI 平台），CLI（AE CLI）是面向写代码 Agent 的通道。

主通道走 MCP：Quick 桌面版直接调用数数分析 MCP，也可以通过Coding Agent调用 CLI / API。

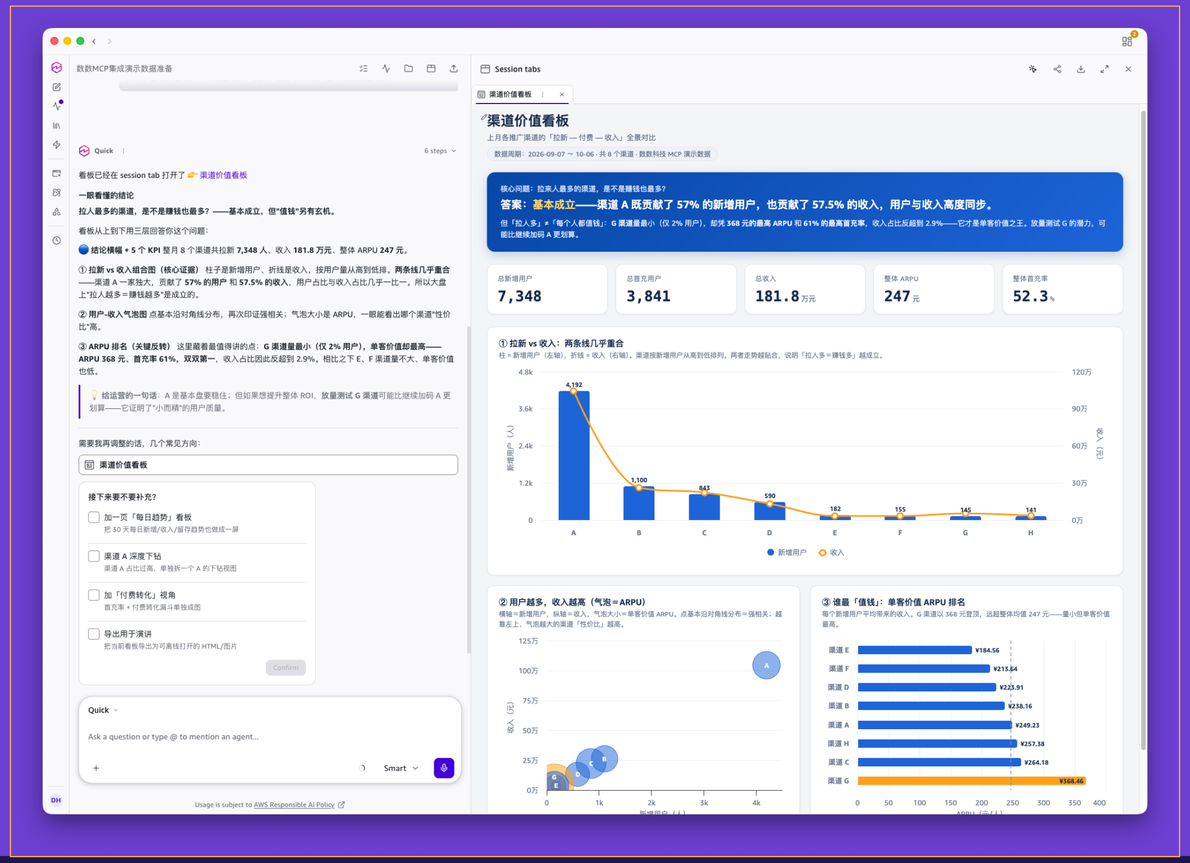
实测①：拉人最多的渠道，赚钱也最多吗？

投放运营

“上个月各渠道的用户和付费都齐了：拉人最多的渠道，是不是赚钱也最多？做个一眼看懂的看板。”

Quick 的结论

基本成立：渠道 A 贡献 57% 新增和 57.5% 收入；但 G 的单客价值最高（ARPU 368 元）。



下一步，可以直接对 Quick 说

「存成 Skill，分享给全组」

「把这个查询逻辑做成 App，分享给全组」

「放进项目组的 Space，同事可以接着追问」

分析是数数 Agentic Engine 算的，口径是数数的；Amazon Quick 把结果交到业务手里，业务接着追问。

实测截图：Amazon Quick 桌面版 × 数数分析 MCP（演示项目数据）。

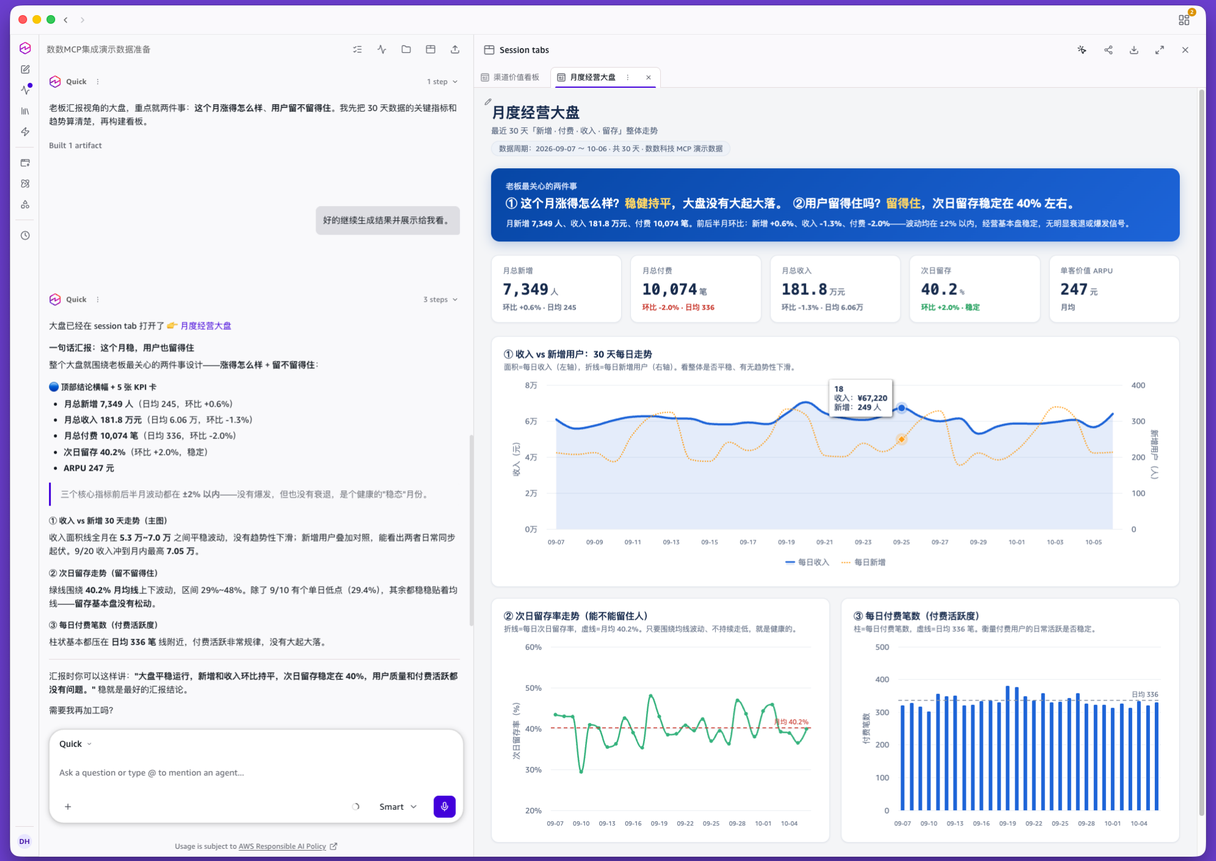
实测②：给老板的月度大盘

运营负责人

“用最近 30 天的新增、付费和留存，做一个给老板汇报的大盘：这个月涨得怎么样，用户留不留得住？”

Quick 的结论

稳健持平，次日留存稳定在 40% 左右。



下一步，可以直接对 Quick 说

「这个查询以后每天早上 9 点跑一次」

「给张总发一封邮件，附上这个大盘」

「按这个思路生成一份汇报 PPT」

实测截图：Amazon Quick 桌面版 × 数数分析 MCP（演示项目数据）。

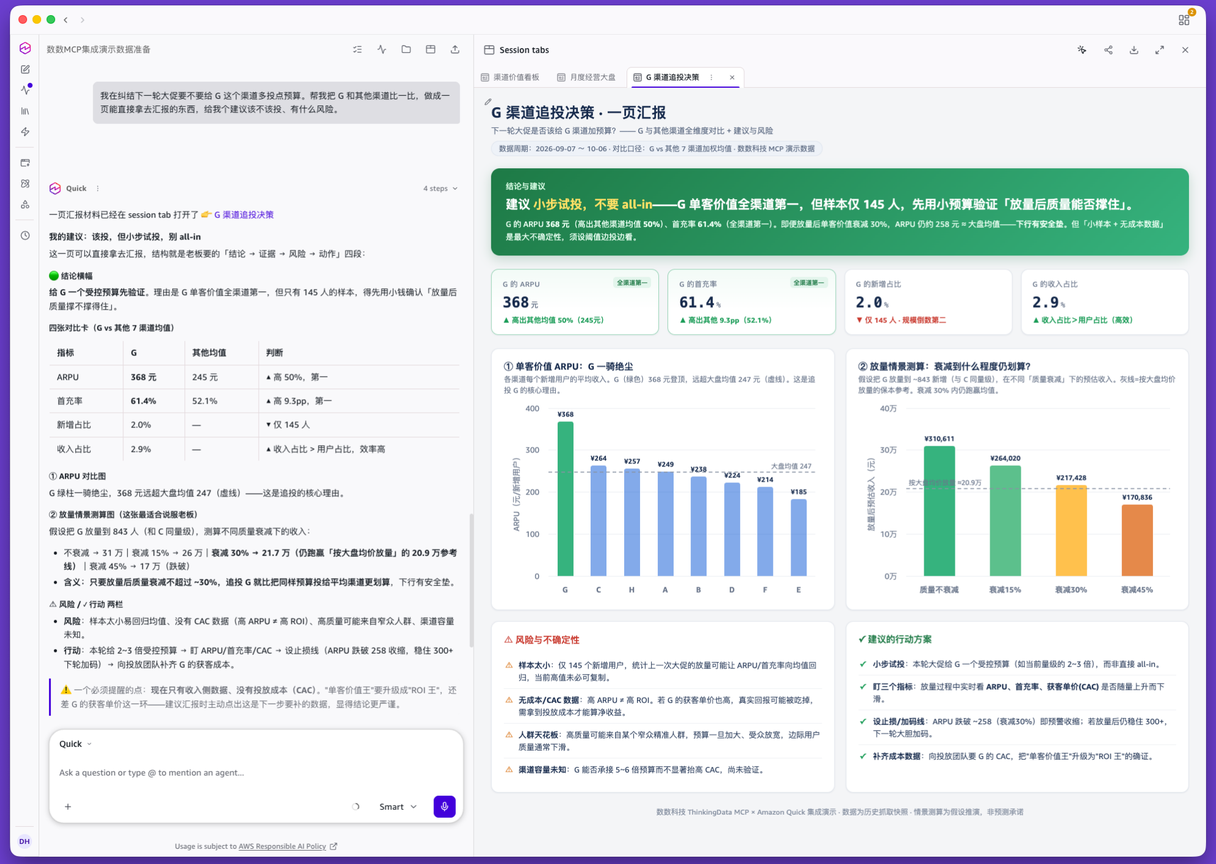
实测③：G 渠道该不该追投？

市场投放

“下一轮大促要不要给 G 渠道多投预算？和其他渠道比一比，做成一页汇报，给建议和风险。”

Quick 的结论

建议小步试投、不要 all-in：G 单客价值第一，但样本只有 145 人。



下一步，可以直接对 Quick 说

「按这个思路写成报告，存到我的 OneDrive」

「把结果发布到 SharePoint 项目组共享文档」

「草拟会议主题，约明早 10 点和项目组开会」

实测截图：Amazon Quick 桌面版 × 数数分析 MCP（演示项目数据）。

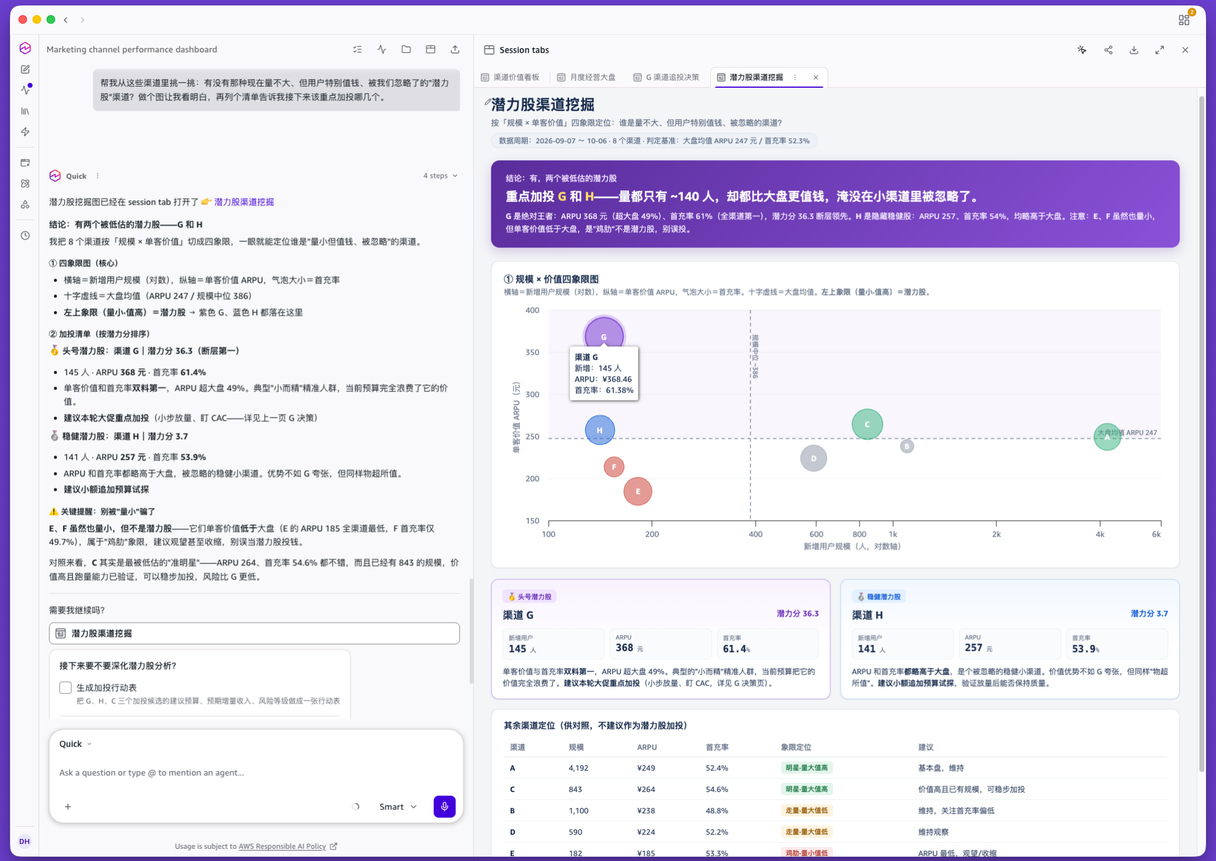
实测④：找出被忽略的潜力股渠道

增长分析

“有没有量不大、但用户特别值钱、被我们忽略的‘潜力股’渠道？做个图，再列个加投清单。”

Quick 的结论

重点加投 G 和 H：量都只有约 140 人，却比大盘更值钱。



下一步，可以直接对 Quick 说

「连接 CRM，看看这些用户背后的业务逻辑」

「在 Jira 给投放组建一个跟进任务」

「把加投清单发到项目组 Slack 频道」

实测截图：Amazon Quick 桌面版 × 数数分析 MCP（演示项目数据）。

不止把图画出来——Quick 的差异在“图之后”（一）：产物是组织资产

真正的分界线发生在结果生成之后：产物属于谁、保存在哪里、能否被组织安全复用。

01

对话生成

session tab 中生成 artifact

02

组织归档

保存到账号云端 Library

03

跨端复用

Web、移动端、桌面均可查看

04

继续工作

从 Library Open in chat

成品形态直接可交付

对话中可以直接生成 Word / PPT / PDF 等文档，不停留在聊天框里。

把图画出来只是起点；让不写代码的人持续使用，让组织安全复用，才是最后一公里。

来源：Library - Amazon Quick；Working with chat；Document and visual creation

不止把图画出来——Quick 的差异在“图之后”（二）：组织级治理 + 与 AWS 身份打通

Amazon Quick Enterprise Edition

组织级平台能力

发布：Skills / Chat agents 带版本，可按角色审批

统一管理：Connectors / MCP 的权限与开关

团队空间：Spaces 组合知识、数据、Dashboard 与行动

交付：Apps 一键发布给团队

自动运行：云端 Routines 按计划执行

AWS 身份与数据治理

身份：IAM Identity Center、AD、Entra ID、Okta

审计：CloudTrail

内容防护：Amazon Bedrock Guardrails

防泄漏：Microsoft Purview DLP

审批：知识库、Spaces、Chat agents 可配置 workflow

隔离：知识库按权限控制

以上治理能力归属 Enterprise Edition 账号，覆盖 Professional 与 Enterprise 两档用户订阅。

面向业务用户交付，同时让组织可治理，并与企业 AWS 身份和数据底座打通。

来源：Amazon Quick Skills、Chat agents、Connectors、Spaces、Apps、Routines、Security、Guardrails、DLP、Approval workflows

04 延伸：行为洞察 → 主动干预

数数 Agentic Engine：让每次干预可验证

从洞察到干预：用实验验证每一次行动



结论回到观察，持续迭代：每一次干预都有实验证据

典型场景

礼包定价 · 首充引导 · 弹窗时机 · push 召回策略

接上实测的结论（示例）

G 渠道小步试投：用 A/B 验证放量后 ARPU 和首充率撑不撑得住

这是运营侧实验

给用户分组、比较策略，验证的是业务干预效果；和 AgentCore harness 页提到的 A/B（面向 Agent 版本迭代）是两件不同的事。

05 场景实践与收尾

从一个场景讲透，到总结两端

跨行业实践：Data Agent 从 PoC 走向规模化的共性挑战

三个共性问题

1. 数据及时性：能不能在变化发生时就看到，不是等隔夜报表
2. 业务口径：同一个指标，各团队算法是否统一
3. 分析到行动的衔接：结论能不能直接变成下一步的动作，而不是停在报告里

游戏行业的例子

口径统一：“值钱”统一按 ARPU、首充率算；及时性：这次用的是固定区间的演示数据，不是实时流；行动衔接：结论直接变成“小步试投 + 盯指标”的动作

行业	可复用场景	主要关注点
OTA	用户流失、价格与转化变化	实时性、跨系统协同
内容	活跃下降、推荐效果变化	人群分层、反馈速度
互联网	活动转化、生命周期运营	实验、触达、效果回收

回到开场：两端都接上，闭环才转起来



左端：企业的 Agent 在 AgentCore 上放心地行动；右端：结果在工作台里被业务看见；中间：数数的数据与行业能力。

三条带走

01 **先划边界，再放权**

Agent 进入生产，首先要解决安全运行与受控执行——身份、策略、审计放到 Agent 代码之外，交给底座

02 **洞察要进业务流**

数据、知识和 Skills 通过 MCP / CLI 进入工作流，才能让分析结果真正被业务使用

03 **从结果走向实验与回收**

从结果呈现继续走向实验、触达与效果回收，行动闭环才真正完成

底座让 Agent 敢行动，最后一公里让行动被看见。

让我们一起继续探索，不断尝试，无限进步 **——期待在未来和各位有更多交流**

THANKS

演讲嘉宾：黄达